
Gobli Auto-Sniper

Setup Guide

Version 1.0

20 September 2026

Contents

1. What this is	3
2. What you need before you start.....	3
3. How it works	3
4. Installation.....	4
5. Configuration	5
6. Creating the Telegram bot	7
7. First run.....	7
8. Going live: the order matters	9
9. Command reference	10
10. Safety features.....	10
11. Limits and honest expectations.....	11
12. Troubleshooting	11
13. Risk.....	13

1. What this is

Gobli Auto-Sniper is a rule-driven automatic sniper for **pump.fun**, controlled entirely from Telegram. You decide in advance what you want to buy. The bot watches the chain for new launches and buys the moment a coin matches one of your rules, or, in watch mode, the moment that coin proves it is attracting real money.

It is **self-hosted and non-custodial**. It runs on your own machine, your private key never leaves that machine, and there is no server in the middle. Nobody else holds your funds or sees your rules.

It buys automatically. Once you switch to live mode, a matching coin is bought without asking you first. There is no per-trade confirmation. Everything in this guide about dry run, caps and pre-flight checks exists because of that.

The bot also detects launches on bonk.fun (Raydium LaunchLab) and reports the ones that match your rules, but it does not buy there by default. See section 11.

2. What you need before you start

Item	Why, and where it comes from
A Helius API key	One key covers the RPC connection, the detection feeds and transaction submission. Get it at <code>dashboard.helius.dev</code> . The free plan works, at reduced speed (section 7).
A dedicated hot wallet	A new Solana wallet used only by this bot. Not your main wallet, and not one used by any other bot. You will need its private key in base58 form.
A Telegram bot	Your control panel. You create it yourself with <code>@BotFather</code> in a few minutes (section 6).
Somewhere to run it	Any machine with Python 3.10 to 3.13 that can stay online. A Linux VPS is recommended.

Why a VPS near the submission endpoint. Speed on a new launch is decided mostly by the physical distance between your machine and the transaction submission endpoint, and no setting in the configuration file can make up for being far away.

3. How it works

Rules

A rule describes what to buy. There are two ways to identify a coin:

Trigger	You give it	When it buys
Creator wallet	One or more wallet addresses	Instantly, the moment that wallet launches anything
Ticker	A symbol, e.g. <code>HELLOWORLD</code>	Instantly, or once the coin clears thresholds you set

A ticker can be matched **exactly** (only that symbol) or by **containing** it (which also catches variations such as HELLOWORLDCOIN or BABYHELLOWORLD). Case and a leading \$ are ignored. Every rule has an expiry date, and can also carry a buy size and a take-profit / stop-loss.

Two timing modes

- **Instant** buys in the same moment the launch is detected. This is the fastest possible entry and the right choice for a creator rule, because who launched the coin is the signal.
- **Watch** notes the launch, then buys only once real SOL has gone into the coin's bonding curve. The bot watches each candidate for a limited time; `/positions` shows how long each watch has left.

Why watch mode exists

Anyone can mint a coin with any ticker for about 0.02 SOL, and people do exactly that on purpose around an anticipated launch. A ticker rule that fires instantly therefore mostly buys **impersonators**.

Fakes almost never attract real money. So "wait until the curve has taken in N SOL" is the filter that separates the real coin from the copies. This is the single most important idea in the product: **for a ticker rule, use watch mode**. The bot warns you when you choose instant timing on a ticker-only rule, and again before arming it.

Why there is no "minimum liquidity" filter

On a launchpad, a liquidity filter is meaningless at the moment of launch. Every pump.fun coin starts from identical virtual reserves with no real SOL in it, so a rule like "at least \$2M liquidity" would simply never fire. The honest measurement is watch mode's **minimum SOL in the curve**: money that buyers have actually put in. You can also set a **maximum** SOL in the curve (the point where you would be buying too late) and a cap on the creator's own share of the supply.

4. Installation

Supported Python versions: **3.10 to 3.13**. The pinned dependencies may not build on newer releases yet.

On a Linux VPS (recommended)

From the product folder, run:

```
bash vps/install.sh
```

This creates the virtual environment, installs the pinned dependencies, locks down `.env`, and runs the layout test to confirm the install is sound.

Manually, or on Windows

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env              # then fill it in (section 5)
chmod 600 .env                    # Linux/macOS
```

Always run the bot inside the virtual environment. The dependencies are pinned to exact versions because this process signs transactions with a live key, so the dependency set is part of what you are trusting. On a machine where newer solders or solana packages are installed globally, running outside the venv lets those shadow the pinned versions.

Keeping it running

To run it unattended on a VPS, install the included systemd service:

```
sudo cp vps/autosniper.service /etc/systemd/system/
sudo nano /etc/systemd/system/autosniper.service # set User= and the paths
sudo systemctl daemon-reload
sudo systemctl enable --now autosniper
journalctl -u autosniper -f                    # watch the log
```

5. Configuration

All settings live in `.env`, copied from `.env.example`. Only four values are required: `HELIUS_API_KEY`, `PRIVATE_KEY`, `TELEGRAM_BOT_TOKEN` and `TELEGRAM_CHAT_ID`. Everything else has a working default. The bot validates every value on startup and refuses to run on a bad configuration.

The wallet must be dedicated. `PRIVATE_KEY` signs automated buys with no human in the loop. Use a hot wallet created for this bot alone, funded only with what you accept losing. Never put your main wallet's key here.

Setting	What it does	Default	Safe to change?
<code>HELIUS_API_KEY</code>	Your Helius key. Covers RPC, detection feeds and Sender.	(empty)	Required
<code>SENDER_REGION</code>	Which Helius Sender endpoint to submit through. Pick the one nearest your machine: <code>slc</code> , <code>ewr</code> , <code>lon</code> , <code>fra</code> , <code>ams</code> , <code>sg</code> , <code>tyo</code> or <code>global</code> .	<code>fra</code>	Yes, set it to match your VPS
<code>HTTP_RPC_URL</code> <code>PREPROCESSED_WS_URL</code> <code>LASERSTREAM_WS_URL</code> <code>HELIUS_SENDER_URL</code>	Optional endpoint overrides. When blank they are derived from your API key.	(empty)	Leave blank unless you know you need them
<code>PRIVATE_KEY</code>	Base58 private key of the dedicated hot wallet.	(empty)	Required

Setting	What it does	Default	Safe to change?
HOT_WALLET_MAX_BALANCE_SOL	The bot refuses to start if the wallet holds more than this. Limits the damage from a mistake, and catches the wrong key ending up in the file.	2.0	Yes, deliberately
TELEGRAM_BOT_TOKEN	Token from @BotFather.	(empty)	Required
TELEGRAM_CHAT_ID	Your numeric Telegram user id. Only this chat gets answers; every other chat is ignored silently.	(empty)	Required
BUY_AMOUNT_SOL	Default SOL spent per buy. A rule can set its own size.	0.05	Yes (also /setbuy)
SLIPPAGE_BPS	Slippage in basis points (2500 = 25%). Set wide on purpose: price moves a lot between detection and landing. Spend is still capped exactly by the buy amount.	2500	Yes (also /setslippage)
TIP_SOL	Helius Sender tip per transaction. 0.001 reaches every routing path. Below 0.000005 SOL, Sender drops the transaction silently.	0.001	Yes, within limits (also /settip)
PRIORITY_FEE_MICRO_LAMPORTS	Priority fee per compute unit.	500000	Yes, carefully
DRY_RUN	true builds, signs and simulates every buy, reports the result and spends nothing. This is the mode the bot starts in after every restart.	true	Leave as true; go live from Telegram
MAX_SOL_PER_SNIPE	Hard cap on any single buy.	0.1	Yes, restart required
MAX_SOL_PER_DAY	Hard cap on total spend per day.	1.0	Yes, restart required
MAX_CONCURRENT_POSITIONS	Maximum number of positions open at once.	5	Yes, restart required
PER_CREATOR_COOLDOWN_SECONDS	How long to wait before buying from the same creator again. Serial rugpulls launch several coins in a row.	300	Yes, restart required

The four risk caps can only be changed in .env followed by a restart. That is deliberate: a limit you can raise from your phone in a hurry is not much of a limit. Changes made with /setbuy, /setslippage and /settip are saved and survive a restart.

6. Creating the Telegram bot

WARNING: CHOOSE A BORING NAME

Give your bot a **boring, unremarkable name and username**. Do **not** use words like "sniper", "solana", "trade" or "pump", and keep the bio equally plain.

Telegram's automated filters have been known to **ban trading bots on the name alone**, before the bot has done anything at all. Something forgettable, such as "Ledger Notes", is the right idea.

1. Open Telegram, message **@BotFather** and send /newbot.
2. Choose a name and a username, following the warning above.
3. BotFather replies with a token in the form 0000000000:XXXXXXXXXXXX. . . . That is your TELEGRAM_BOT_TOKEN. Treat it like a password: anyone with it can impersonate your bot.
4. Message **@userinfobot** and send anything. It replies with your numeric user id. That is your TELEGRAM_CHAT_ID.
5. Open a chat with your new bot and press **Start**. Until you do, Telegram does not allow the bot to message you.

Only that one chat id ever gets a reply. Messages from anyone else are ignored silently, so someone who finds your bot learns nothing about whether it is running.

Optional: send BotFather /setcommands, choose your bot and paste the list below so the commands appear in Telegram's menu.

```
start - Build a new snipe rule
rules - List rules
positions - Open positions and watches
stats - Feed and latency numbers
config - Current settings
balance - Wallet balance
preflight - Pre-flight check against a real coin
setbuy - Set default buy amount
dry - Simulate only
live - Spend real SOL
panic - Disable everything and sell out
help - All commands
```

7. First run

Fund the wallet

Send the hot wallet only what you accept losing to an automated strategy, and keep it below HOT_WALLET_MAX_BALANCE_SOL, or the bot will refuse to start.

Start it

```
bash vps/start.sh          # or: python main.py
```

If anything in `.env` is missing or invalid, the bot prints the full list of problems and exits, rather than running on a bad configuration. Fix everything on the list and start it again.

Once it is running, it sends a startup message to your Telegram chat. The key thing to read there is **which detection feeds are live** on your Helius plan. It starts in **dry run**; send `/config` at any time to confirm the mode, wallet, balance, settings and caps.

Which feeds your Helius plan unlocks

The bot subscribes to every feed your plan allows and acts on whichever reports a launch first. A cheaper plan means later entries, not a broken bot.

Feed	Stage	Plan needed
preprocessedSubscribe	Before execution, roughly 8 ms ahead of other feeds	Any paid plan (Developer and up)
transactionSubscribe	After execution	Any paid plan
logsSubscribe	After execution, from pump.fun's event log	Free plan

On the free plan only the logs feed runs. It still detects every launch, with the coin's exact reserves, but it is **the slowest of the three**, so you will be entering behind anyone on a paid feed. It is fine for dry run and testing; upgrade before competing for entries. Check Helius for current plan pricing.

Reading /stats

Line	What it tells you
Feeds	Each feed shows connected or disconnected, messages received and how many times it was first to report a launch. A feed your plan does not include shows as unavailable , with the reason. If two feeds are running, you also see how much later the second one arrives on average.
Pipeline	Launches seen, launches decoded (with decode failures), and how many matched one of your rules.
Buys	Buys attempted, landed and failed; dry runs that passed and that would have failed.
Median detect → sent	Milliseconds from detecting a launch to submitting your buy. This is the part your machine controls.
Median slot delta	How many slots after the launch your buys land. Lower is better.
Creator vault corrected	An internal counter. Include it if you ever report a problem.
Spent today	SOL spent today against <code>MAX_SOL_PER_DAY</code> .

Building a rule

Send `/start`. The bot asks whether you have the **creator's wallet** or only the **ticker**, then walks you through the rest one message at a time. Send `skip` to leave an optional threshold unset, and `/cancel` to abandon the wizard.

- **Creator rule:** wallet address(es), comma-separated or one per line → maximum creator buy % → buy size → expiry → take profit / stop loss. Creator rules always fire instantly.
- **Ticker rule:** ticker(s) → exact or containing match → watch or instant → (watch only) minimum SOL in the curve, then maximum SOL → maximum creator buy % → buy size → expiry → take profit / stop loss.

Expiry is written as 6h, 3d, 2w or a date such as 2026-10-01. Take profit / stop loss is written as two percentages, e.g. 100/40 for +100% and -40%. For watch mode, a minimum of 5 to 30 SOL in the curve is a reasonable starting range. You see a summary before anything is armed, and the rule is live only when you press **Arm it**.

8. Going live: the order matters

Do these three steps in order. Each one proves something the previous one cannot.

Step 1: run the pre-flight check

```
/preflight <mint address>
```

Use any pump.fun coin that is still on its bonding curve; take one from pump.fun's front page. The bot reads that coin's real on-chain state, builds and signs the actual buy instruction for it, and runs it through the network's transaction simulator with signature verification on. **Nothing is sent and no SOL moves.**

This is what proves the chain accepts the buy instruction. The bundled layout test proves the instruction matches pump.fun's published program definition, which is not the same claim. If the pre-flight check fails, **do not go live:** keep the whole message, because it is the diagnostic a fix will be based on.

Step 2: watch it in dry run

Arm a rule and let it catch a few real launches. In dry run, every matched launch is still built, signed and simulated against the real network. It is not skipped: the bot runs the same transaction it would send, and tells you whether it would have succeeded. A pass in dry run therefore means something. `/stats` counts dry runs that passed and those that would have failed.

Step 3: `/live confirm`

Set a small buy size first (`/setbuy 0.01`, for example), then send `/live`. The bot shows your wallet and balance and asks you to send `/live confirm`. From then on, matching buys spend real SOL with no confirmation per trade. Send `/dry` to return to simulation at any time.

Live mode is not saved. After any restart the bot comes back in the mode set by `DRY_RUN` in `.env`, which is dry run by default, and you must send `/live confirm` again.

9. Command reference

Command	What it does
<code>/start</code>	Build a new rule with the step-by-step wizard.
<code>/cancel</code>	Abandon the wizard. Nothing is armed.
<code>/rules</code>	List every rule with its id, status (armed, paused, expired or done) and timing.
<code>/rule_on <id></code>	Arm a paused rule.
<code>/rule_off <id></code>	Pause a rule without deleting it.
<code>/delete <id></code>	Delete a rule.
<code>/positions</code>	Open positions with SOL in and current profit/loss, plus coins being watched with SOL in the curve and time left.
<code>/stats</code>	Feed status, pipeline counts, buy results, latency and today's spend.
<code>/config</code>	Mode, kill switch state, wallet, balance, trading settings, caps and Sender region.
<code>/balance</code>	Wallet address and SOL balance.
<code>/preflight <mint></code>	Build, sign and simulate a real buy for that coin without sending it. Run this before going live. <code>/testbuy</code> is an older alias.
<code>/setbuy <sol></code>	Set the default buy size. Cannot exceed <code>MAX_SOL_PER_SNIPE</code> . Survives a restart.
<code>/setslippage <bps></code>	Set slippage in basis points, 1 to 10000. Survives a restart.
<code>/settip <sol></code>	Set the Sender tip per transaction. Values below Sender's minimum are refused. Survives a restart.
<code>/dry</code>	Switch to dry run: simulate and report, spend nothing.
<code>/live, then /live confirm</code>	Switch to live mode. Does not survive a restart.
<code>/panic</code>	Kill switch: disable every rule, drop every watch and sell every position the bot opened.
<code>/resume</code>	Release the kill switch. Rules stay paused until you re-arm each with <code>/rule_on</code> .
<code>/help</code>	Show the command list.

10. Safety features

- **Kill switch.** `/panic` disables every rule, drops every watch and sells every position the bot opened. After `/resume`, rules stay paused until you re-arm them one by one, so nothing fires by surprise.
- **Caps.** Per snipe, per day, and on concurrent positions. They are checked immediately before each buy, not only when a rule is armed, because the situation can change after you set a rule up. Raising them requires editing `.env` and restarting.
- **Daily spend survives a restart**, so a crash loop cannot reset the daily cap.

- **Creator cooldown.** After buying from a creator, the bot waits before buying from them again (5 minutes by default).
- **Never buys the same coin twice.**
- **Hot-wallet ceiling.** The bot refuses to start if the wallet holds more than `HOT_WALLET_MAX_BALANCE_SOL`.
- **Every rule must expire.** A forgotten rule that fires months later is how people lose money in their sleep.
- **Impersonation guards.** Tickers mixing alphabets (a Cyrillic letter inside a Latin ticker, for example) are rejected, and ticker-only instant rules come with explicit warnings.
- **Live mode does not survive a restart.** The bot always comes back in the mode set in `.env`, dry run by default. An unexpected restart never leaves it spending unattended in a state you did not just confirm.
- **Single-owner control.** Only your chat id gets a response; everything else is ignored.

11. Limits and honest expectations

- **pump.fun bonding curves only.** Once a coin graduates off its curve to PumpSwap or Raydium, this bot can no longer trade it. You must sell it manually from your wallet.
- **bonk.fun is detection-only.** bonk.fun launches that match your rules are detected and reported, but buying there is off by default. Many bonk.fun pools are also quoted in tokens other than SOL, and those are skipped.
- **Positions are tracked in memory.** A restart loses position tracking and any armed take-profit or stop-loss. The tokens stay in your wallet; the bot simply stops watching them. Rules, caps and daily spend *do* persist.
- **Some buys fail on-chain.** A small share of buys fail for reasons outside the bot's control, such as the price moving past your slippage before the transaction lands. These cost fees, not positions.
- **Safety checks are the fast ones only:** what can be read from the launch transaction itself. There is no holder-distribution, socials or third-party risk scoring.
- **Coin metadata is never fetched.** Loading it takes seconds and the whole budget is milliseconds, so matching uses the ticker recorded in the launch transaction.
- **Latency figures are only as good as the feed** that reports them, and have not yet been benchmarked against live mainnet.
- **pump.fun changes.** pump.fun updates its program every few months. When it does, the bot may need an update before it can buy again (section 12).

12. Troubleshooting

Symptom	Cause	Fix
Bot exits on startup with a list of problems	Config validation failed: a required <code>.env</code> value is missing or malformed.	Fix every item on the list, then start again. It reports all problems at once.
Bot refuses to start over the wallet balance	The wallet holds more than <code>HOT_WALLET_MAX_BALANCE_SOL</code> , or the wrong private key is in <code>.env</code> .	Check the key is the dedicated hot wallet. Move surplus SOL out, or raise the ceiling deliberately.

Symptom	Cause	Fix
Bot never replies in Telegram	TELEGRAM_CHAT_ID is wrong, you never pressed Start in the bot's chat, or you are writing from a group.	Re-check the id with @userinfobot, press Start, message the bot in a private chat.
A feed shows unavailable in /stats	Your Helius plan does not include that feed. On the free plan only logsSubscribe runs.	Expected on the free plan. Upgrade to a paid plan to unlock the faster feeds.
A feed shows as disconnected	Network trouble, or a problem with the Helius key.	Check the key and the machine's connectivity; read the log (journalctl -u autosniper -f).
Buy or pre-flight fail for lack of funds	The wallet does not hold enough SOL for the buy plus tip and fees.	Top up the wallet, staying under the hot-wallet ceiling, or lower the buy size.
Pre-flight or a sell fails on a coin that already graduated	The coin has left its bonding curve; the bot only trades bonding curves.	Test with a coin still on its curve. Sell graduated coins manually.
/preflight fails on a coin still on its curve	The chain is rejecting the buy instruction, often after a pump.fun update.	Do not go live. Keep the full message and run the layout test (below).
/config shows "fallback constants" for pump.fun Global	The bot could not read pump.fun's live global settings.	Check the RPC connection and key; watch the log.
/setbuy or the wizard refuses a buy size	The amount exceeds MAX_SOL_PER_SNIPE.	Use a smaller amount, or raise the cap in .env and restart.
/settip refuses a value	Below Helius Sender's minimum; the transaction would be dropped silently.	Use 0.001.
Import errors or unexpected package versions	Running outside the virtual environment, so global packages shadow the pinned ones.	Activate the venv, or use vps/start.sh.

After a pump.fun update

A wrong account list does not fail on your machine; it fails on-chain, after the snipe is already lost. After any pump.fun announcement, run the layout test:

```
python tests/test_layout.py      # or: pytest tests/
```

Then run /preflight against a live coin before trusting live mode again. The channel t.me/pump_tech_updates announces breaking changes.

13. Risk

Sniping new launches loses money often, and an auto-sniper loses it without asking you first.

Most new tokens are worthless, and many are deliberate frauds. Transactions can fail after the fees are paid. Liquidity can vanish faster than any exit can fire.

Fund the hot wallet only with what you can afford to lose entirely. Treat `MAX_SOL_PER_DAY` as the amount you are willing to lose in a day, not as a target.

This guide makes no promise of profit or returns. Read the `LICENSE` file included with the product, in particular Section 4, before funding anything; it contains the full terms.